

Centro de Estudios Superiores del Noroeste.

Ing. En Desarrollo de Software



Actividad

Avance 1.

Presenta:

Serrano Silva Jose Angel.

Instructor:

JOSÉ ALFREDO MARTÍNEZ COSÍO

Materia:

Arquitectura de Software

Tijuana, B.C

Junio 2026

1. Descripción y Objetivos

- Contexto: El sector financiero exige confianza absoluta y cero margen de error en el movimiento de dinero.
- Objetivo del MVP: Construir un motor de transferencias internas que garantice la exactitud de los saldos (ACID) y resista caídas de infraestructura.
- Expectativa del Usuario: Transacciones instantáneas. Si el sistema falla, espera un mensaje claro, nunca un error de código.

2. Atributos de Calidad

- Consistencia
 - Definición e Impacto: Es la garantía de que el sistema transita de un estado válido a otro, sin pérdida ni generación espontánea de fondos. Es vital para el MVP porque un solo error de saldo negativo o duplicidad destruye la viabilidad del negocio.
 - Escenario de Arquitectura: Se pone a prueba mediante peticiones concurrentes. El sistema lo resuelve a nivel de base de datos MySQL utilizando control de concurrencia mediante un campo versión y transacciones estructuradas con START TRANSACTION y COMMIT/ROLLBACK.
- Disponibilidad y Resiliencia
 - Definición e Impacto: La capacidad del sistema para seguir operando, o fallar con gracia, cuando un componente crítico colapsa.
 - Escenario de Arquitectura (La "Caja Rota"): Si el contenedor de la base de datos MySQL sufre una desconexión total de 10 minutos, el atributo de disponibilidad se protege mediante el patrón Circuit Breaker. En lugar de que el backend en Python acumule peticiones hasta saturar la memoria, el circuito se abre. Durante esos 10 minutos, el microservicio intercepta las peticiones y responde instantáneamente al API Gateway con un HTTP 503 Service Unavailable. El frontend móvil captura este código y muestra una alerta amigable: "Demoras técnicas. Tu dinero está seguro, intenta en unos minutos", protegiendo la confianza del usuario y los recursos del servidor.

- **Mantenibilidad y Testabilidad**
 - **Definición e Impacto:** La facilidad con la que el código puede ser modificado, auditado y probado de forma automatizada. En el sector Fintech, el código debe ser auditable.
 - **Escenario de Arquitectura:** Se evalúa al intentar agregar una nueva regla. Gracias a la separación de responsabilidades, el equipo puede programar y ejecutar pruebas unitarias sobre el núcleo de la transferencia simulando la base de datos, sin necesidad de levantar el contenedor de MySQL.

3. Propuesta de Estilo y Patrón Arquitectónico

- **Estilo Arquitectónico Seleccionado:**
 - Se selecciona una Arquitectura Distribuida basada en Microservicios a nivel macroscópico separando Transferencias, Préstamos y Notificaciones, complementada con Arquitectura Hexagonal a nivel interno para el diseño del código de cada servicio. Además, se incorpora un enfoque orientado a eventos para tareas asíncronas.

Justificación de Ingeniería:

- **Aislamiento de Dominios (Microservicios):** Al encapsular el módulo de transferencias en su propio contenedor, se asegura que un fallo en el área de préstamos no detenga el flujo de dinero. Además, permite asignar recursos de cómputo (CPU/RAM) de forma independiente a los servicios más demandados.
- **Protección de Reglas de Negocio (Hexagonal):** En una app financiera, la lógica matemática y de validación es el activo más valioso. La arquitectura hexagonal garantiza que estas reglas no se acoplen a las dependencias externas. Si en el futuro se migra la base de datos, las reglas de la transferencia permanecen intactas.
- **Desacoplamiento de Procesos (Eventos):** Integrar un Message Broker permite que el servicio principal responda al usuario en milisegundos tras hacer el COMMIT en la base de datos, dejando que el envío de correos o notificaciones push ocurra en segundo plano. Esto mejora drásticamente el rendimiento percibido sin sacrificar la confiabilidad.